1 WRITING CLEAN CODE

In order to make your code readable you have to clean your code regularly. This step is very important to not to slow down the programming process in the future programming. You will probably spent the same amount of time cleaning the code that you needed for writing a working version.

In general you can follow some rules:

1. Use FUNCTIONS for every single action,
2. COMMENT the code only where is necessary,
3. Use EXPLANATORY CONSTANTS and VARIABLES

to make your code clean.

1.1 Tasks:

1. Change your code to look like:

Program 1: Clean code of moving forward program.

```
void setup()
{
  setIOpins();
}

void loop()
{
  moveForward();
  delay(3000);
  robotStop();
}
```

2. Write programming functions for moving the robot in several dirrections:

    1. `moveForward()`,
    2. `moveLeft()`,
    3. `moveRight()`,
    4. `moveBackward()`,
    5. `robotStop()`.

3. Save all the functions into header file: `RobotMovingFunctions.h`

1.2 Questions:

1. <++>
2. <++>
3. <++>

1.3 CLEAN CODE EXPLAINED

1.3.1 Comments - YES and NO

Comments are very helpful and necessary. Keep them short and meaningful whenever is needed. May also help during thinking process while beginning designing the code.

```
  // robot will go forward
  digitalWrite(7,HIGH);
  digitalWrite(6,LOW);
  digitalWrite(5,HIGH);
  digitalWrite(4,LOW);
```

Don't use comments where the code is self-explanatory, for example:

```
    delay(3000); //wait for 3000ms
```

1.3.2 Functions

Concatenate programming code into meaningful functions is a must! Previous example of code for `driving a robot forward` is very difficult to understand at first sight. We can make cleaner code as is shown in nex example where is easier to understand what-is-what:

```
  void robotForward()
  {
    digitalWrite(LEFT_MOTOR_PIN_1,HIGH);
    digitalWrite(LEFT_MOTOR_PIN_2,LOW);
    digitalWrite(RIGHT_MOTOR_PIN_1,HIGH);
    digitalWrite(RIGHT_MOTOR_PIN_2,LOW);
  }
```

Compact code is more understandable than large one, see next example:

```
void setup()
{
  setIOpins();
  robotForeward();
  delay(3000);
  robotStop();
}
```

1.3.2.1 Function declaration Function declaration is highly advisable since allow you a quick overview of available functions in a current file. It is like a table of functions with it's return type and parameters. All declarations are tipically found at the beginig of the file.

```
void moveForward();
void moveLeft();
void moveLeft_PWM(int pwm_value);
```

1.3.2.2 Function Definition A function definition provides the actual body of the function.

```
void robotForward()
{
  digitalWrite(LEFT_MOTOR_PIN_1,HIGH);
  digitalWrite(LEFT_MOTOR_PIN_2,LOW);
  digitalWrite(RIGHT_MOTOR_PIN_1,HIGH);
  digitalWrite(RIGHT_MOTOR_PIN_2,LOW);
}
```

1.3.3 Constants

Use explanatory constants to more clearly represent unintuitive numbers or other abstract values. Use these constants instead of comments since these numbers will appear several times during programming code.

```
const int LEFT_MOTOR_PIN_1 = 7;
const int LEFT_MOTOR_PIN_2 = 6;
```

Now you can easily see why the pins are set as OUTPUT. Because there is `Left motor` attached.

```
void setIOpins()
{
  pinMode(LEFT_MOTOR_PIN_1, OUTPUT);
  pinMode(LEFT_MOTOR_PIN_2, OUTPUT);
}
```

1.3.4 Variables

Use explanatory variables to make if-statements easily readable and thus understandable. Make `boolean` variables as short statements with no inverting logic.

For example we will use the case where the robot should stop as soon it hits the obstacle with front bumper. The worst case scenario of the program could look like this (we have all done it at some point):

```
void loop()
{
  if (digitalRead(A0) == FALSE){
    digitalWrite(7, HIGH);
    digitalWrite(6, LOW);
    digitalWrite(5, HIGH);
    digitalWrite(4, LOW);
  }else{
    digitalWrite(7, LOW);
    digitalWrite(6, LOW);
    digitalWrite(5, LOW);
    digitalWrite(4, LOW);
  }
}
```

And more clean representation of same functionality is shown in next example of the code. Line 3 is easy readable, simple, clear and easy understandable.

```
1 void loop()
2 {
3     int front_bumper_is_pressed = digitalRead(BUMPER_INPUT);
4     if (front_bumper_is_pressed) robotStop(); else robotForward();
5 }
```

1.3.5 Header files

To keep our main program file short and transparent as possible we can put supporting code (e.g. functions, settings, ...) into separate file and include it at the beginning of the main program. These files are called header files. We can write a function and save it into header file called “calculate.h”

```
int sumTwoNumbers(int A, int B)
{
  return A+B;
}
```

In our main program we can include the header file and use the function:

```
#include "calculate.h"

int main()
{
  int a = 5, b = 3;
  int sum = sumTwoNumbers(a, b);
}
```

In this way our main code is clean and transparent.

1.3.6 Pre-process

The preprocessors are the directives, which give instructions to the compiler to pre-process the information before actual compilation starts (e.g. `#include` is one of them). You can easily use as such text substitutions for more clear code reading.

```
#define LEFT_MOTOR_PIN_1 7
#define LEFT_MOTOR_PIN_2 6
```

Remember! `#define` is really a simple text substitution and is not type-safe. Furthermore, we have to be certain that our definition will not interfere with other code used outside of our scope e.g. `libraries`. The last example is not the best representation of `#define` usage. In these case the `const int` is more proper way to go (allowed type checking, debugging). But `#define` has other benefits where `const` can not be used.

1.3.6.1 Translations The substitutions can be used as a translation and simplification of code. Such code can be introduced to very young children to get involved in programming.

```
#define vkljuci_led digitalWrite(13,HIGH)
#define izkljuci_led digitalWrite(13, LOW)
#define pocakaj(time) delay(time)
void loop(){
  vkljuci_led;
  pocakaj(1000);
  izkljuci_led;
  pocakaj(1000);
}
```

1.3.6.2 Debugging You can even substitute function names e.g. `debug(txt)` with `Serial.println(txt)` and easily separate debugging code lines from necessary serial print of data.

```
#define debug(txt) Serial.porintln(txt)
void setup()
{
  Serial.begin(9600);
  debug("Running...")
}
void loop()
{
  unsigned long myTime = millis();
  Serial.println(myTime);
  delay(1000);
}
```

When we are done with programming and debugging is not needed anymore we can simply change `#define` line to nothing:

```
#define debug(txt)
```

And these programming sentences will not be used. More sophisticated example is shown where programmer can switch between debugging mode (with `#define DBG 1`) and normal operation (with `#define DBG 0`) where code statement `debug("Running...")` will not even compile into program.

```
#define DBG 1
#if DBG == 1
#define debug(txt) Serial.porintln(txt)
#else
#define debug(txt)
#endif
void setup()
{
  Serial.begin(9600);
  debug("Running...")
}
```

1.4 Summary:

1.5 Issues:

1.5.1 What is the difference between `const int` and #define?

`#define` is textual replacement, so it is as fast as it can get. Also it can save some RAM. The downside is that it's not type-safe.

> **`const`** variables may or may not be replaced inline in the code. It is guaranteed to be type-safe though since it carries its own type with it.